Old Milker Controller Conversation

Project: Pulse / PulseCore / Pluto 9000 vacuum-pulse controller

Created: June 7, 2026. This PDF includes the visible user/assistant conversation and the three photos shared in the chat.

User Message

old controller pics I'm sharing the circuit board for an original milker. no control over anything other than motor power. the only solenoid is for the vac and pulse on their own timing. the potentiometer is not good because it only controls power to the motor, the motor squeals and is useless until it gets to at lease 75% power. What are these chips doing. and would there be anyway to modify the solenoid timing on this old prototype that was purchased to begin to investigate this project. what else should I know? and what else can we learn to do or not to do.

Uploaded Photos

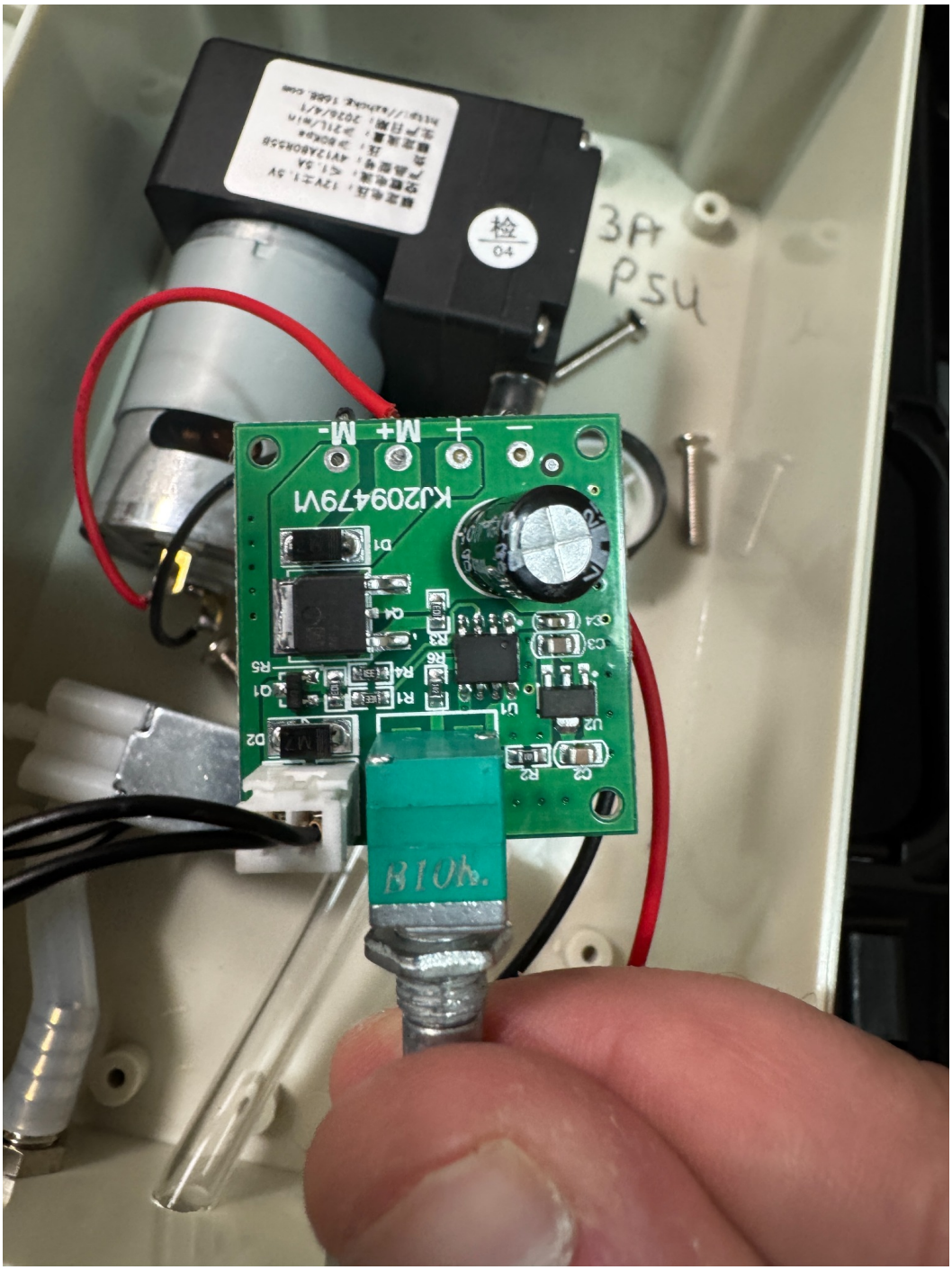

Photo 1 - front of controller PCB with pump and solenoid visible

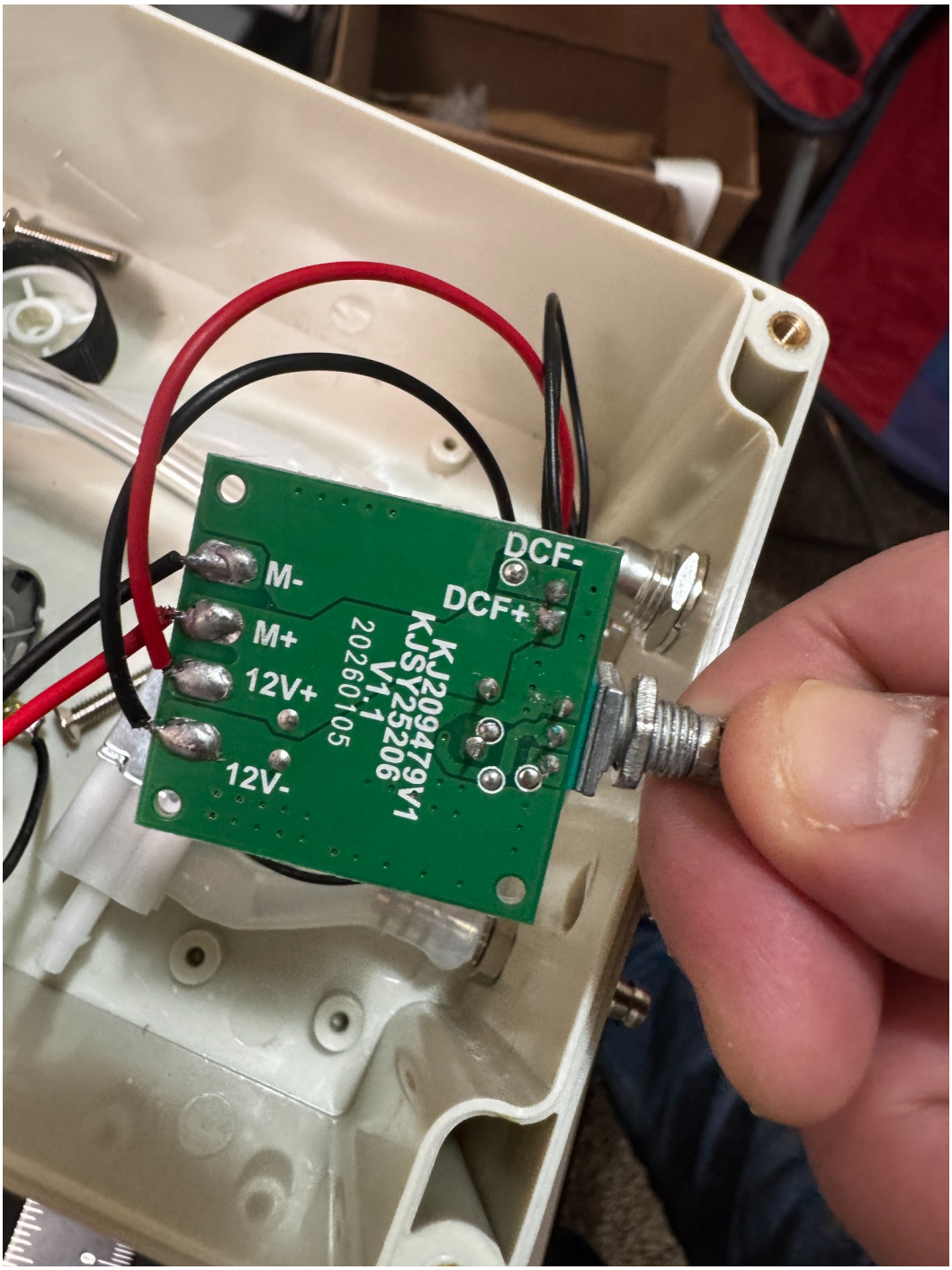

Photo 2 - back of controller PCB showing labels M+/M-, 12V+/12V-, DCF+/DCF-

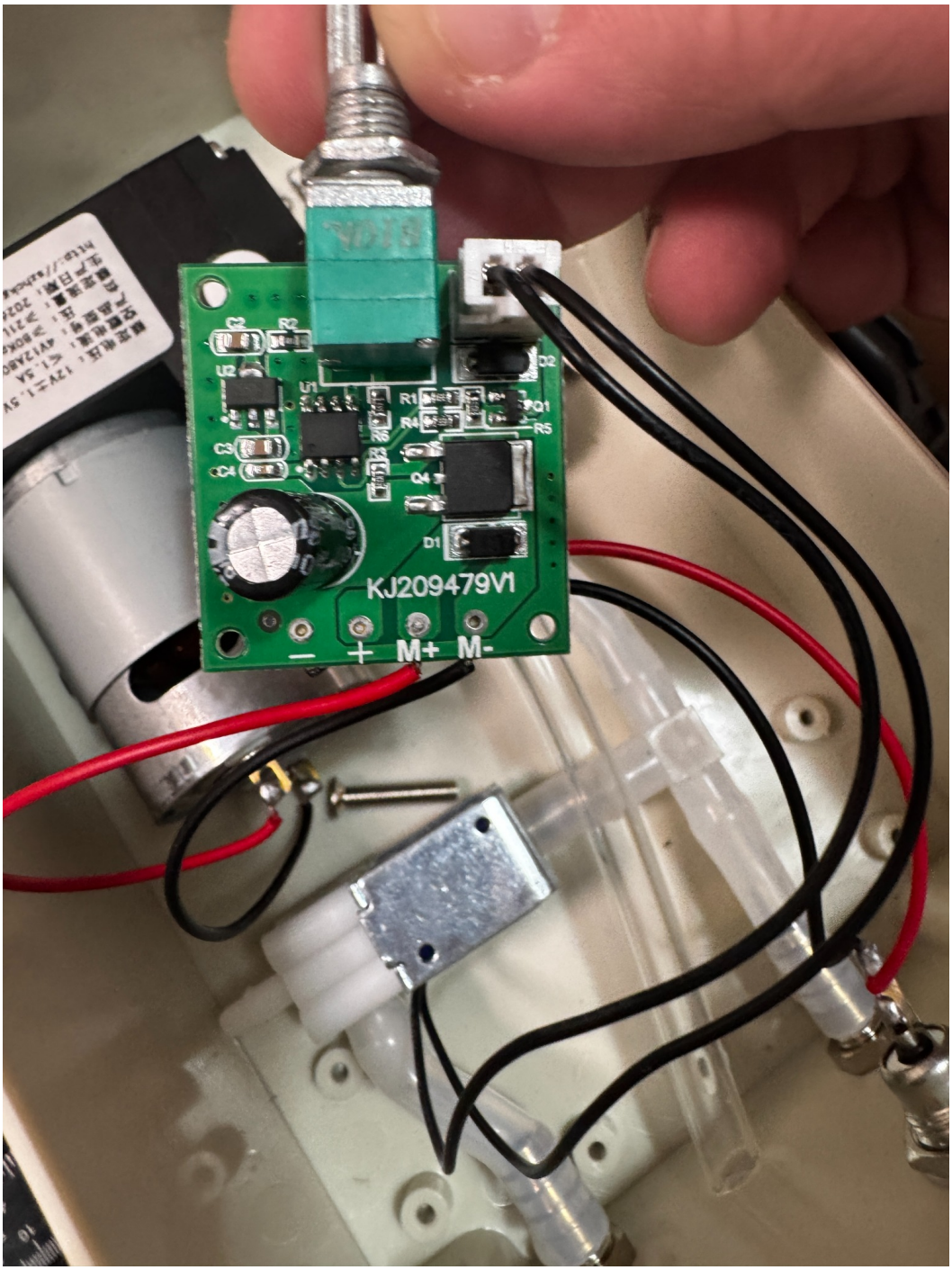

Photo 3 - front of controller PCB showing B10K potentiometer and output connector

Assistant Response

Yes - this board is exactly the kind of "cheap simple controller" we should learn from, not copy.

What this old board is doing

From the labels and layout, it looks like this board has only a few jobs:

- Take 12V power in: Board pads say 12V+ / 12V-. That powers the whole machine.
- Control the pump motor: Board pads say M+ / M-. The big black power device near the motor side is most likely a MOSFET/transistor that chops power to the motor. The B10K potentiometer is almost certainly feeding a control signal to the board so it can reduce or increase motor power.
- Fire the solenoid on a fixed pulse pattern: The back side says DCF+ / DCF-. "DCF" is likely the solenoid/air valve output. The little chip is probably creating the timing pattern. The solenoid is being switched by a small transistor/MOSFET, with a diode nearby to absorb the kickback when the solenoid turns off.
- Basic power cleanup: The round capacitor smooths the 12V supply. The small diodes protect the board from motor/solenoid voltage spikes. One small chip may be a regulator or timing/control chip.

What the chips probably are

I cannot read the chip markings clearly enough to name exact part numbers, but functionally this is probably what they are:

Part	Likely job
Large black power part	Switches/chops power to the motor
8-pin chip	Timer/controller for motor PWM and/or solenoid pulse timing
Small 3/5/6-pin chip	Voltage regulator or small driver stage
Diodes marked D1/D2	Flyback protection for motor/solenoid
Big capacitor	Smooths power and reduces electrical noise
B10K potentiometer	User speed/power input

In plain English: one chip makes the decisions, the bigger black parts do the actual switching.

Why the motor squeals and does nothing useful below 75%

That tells us a lot.

This board is probably not giving the pump a nice controlled speed range. It is likely doing cheap PWM or crude voltage control. At low settings, the motor gets weak chopped power. The motor tries to move, stalls, squeals, vibrates, and does not create useful vacuum.

That means the knob is not really "vacuum control." It is just weak motor power control.

For our PulseCore / Pluto 9000 design, this teaches us: Do not make 0-100% mean raw motor voltage.

A better control map would be something like:

- 0% = pump off

- 1-30% = maybe not used, or mapped to short bursts
- usable vacuum range starts around where the motor actually runs cleanly
- pump should get a strong enough start pulse
- then control vacuum with smarter timing, release, and valve behavior

So instead of letting the pump squeal at 20%, our app should probably hide that useless range from the user.

Can we modify the solenoid timing on this old board?

Maybe, but probably not in a clean way.

There are three possible situations:

1. The timing is inside a programmed chip

This is most likely. If the little chip is a small microcontroller, then the pulse timing is stored in firmware. In that case, there is no simple resistor to change. You would basically have to reprogram or replace the chip. That is usually not worth it.

2. The timing is made by resistors/capacitors

Possible, but less likely. If it is a timer-style circuit, then changing certain resistors or capacitors could change pulse speed or hold time. But we would need to trace the board carefully and identify which parts set the timing. That would be a messy mod.

3. Bypass the board's solenoid control

This is the best experimental option. You could leave the old board powering the motor, but disconnect the solenoid from DCF+ / DCF- and run that solenoid from your own controller/MOSFET.

That would let an ESP32/CoreS3 control:

- pulse speed
- pulse hold
- release time
- duty cycle
- test firing
- different modes

This is basically what our new design is already meant to do.

What I would not do

I would not spend too much time trying to modify this board directly.

Reasons:

- no labeled timing adjustment
- no easy display or feedback
- motor control is already bad
- solenoid timing is probably fixed in firmware

- board is cramped and cheap
- modifying it could break it without teaching much more

This board is useful as a reference, not a platform.

What we can learn from it

The big lessons are:

1. Pump speed control alone is not enough

The old design acts like: "turn the motor weaker or stronger." That is not the same as comfort control.

2. The pump needs a usable minimum power

Since it squeals until around 75%, our future control should avoid low useless power levels.

3. Solenoid timing needs to be adjustable

The old board probably has one hidden factory timing pattern. That is exactly what we are trying to improve.

4. One solenoid can make a basic pulse, but it is limited

One solenoid can create a simple vacuum/pulse effect, but it does not give us the same control as separate valves for:

- main vacuum
- pulse behavior
- release/venting
- comfort control

5. The old design has no real intelligence

No sensor feedback. No modes. No screen. No diagnostics. No manual test page. No independent controls. That confirms we are right to build the new system around the CoreS3 instead of copying this.

What I would do next with this old prototype

Use it as a test mule:

- Measure the solenoid coil voltage while running.
- Measure the timing at DCF+ / DCF-.
- See how long the valve is ON vs OFF.
- Measure motor voltage/current at different knob positions.
- Find the lowest setting where the pump actually runs cleanly.
- Record the old pulse rhythm as a baseline.
- Then beat it with our own controller.

The most important thing to learn from this board is simple: Cheap motor power control feels bad. Fixed solenoid timing is too limited. Our design needs real control over pump behavior, pulse timing, hold, and release.